

Coffee cApp UniPA

Modulo 3

Assegnato: 9 dicembre

Consegna: 29 dicembre

In questo assignment darete vita alla vostra applicazione implementando le componenti di backend e persistenza e un nuovo servizio per il monitoraggio. Iniziate definendo il modello dei dati relazionale per l'applicazione principale.

Di seguito la descrizione del servizio di monitoraggio e le funzionalità da implementare per le 4 diverse tipologie di utenza: **schermo distributore, cliente, addetto manutenzione, gestore del sistema** per comodità ho lasciato in verde le richieste degli assignment precedenti.

Nuovo servizio di monitoraggio

Si tratta di una nuova applicazione, questa volta basata su tecnologia JakartaEE, che utilizza un secondo database relazionale (ma se volete potete anche usare un noSQL) per memorizzare lo stato di funzionamento (attivo, in manutenzione, guasto) e la posizione di tutti i distributori. Il database di questo servizio, essendo indipendente dal database principale, va sincronizzato all'occorrenza. Questo servizio, quindi esporrà dei controller per aggiungere, rimuovere, attivare o mettere in manutenzione un distributore.

Questo servizio avrà anche un controller che riceve i segnali di "heartbeat" dei distributori e uno che produce la lista delle coordinate e dello stato di tutti i distributori che potrebbe essere usato dal Cliente o da altre applicazioni, ad esempio, per visualizzare i distributori su una mappa. Quest'ultimo controller marcherà come guasto un distributore che non è in manutenzione e non ha inviato heartbeat negli ultimi tre minuti (per non sovraccaricare il sistema si potrebbe pensare di verificare la presenza di distributori guasti solo periodicamente, ad esempio, quando il controller riceve la richiesta potrebbe verificare quando è stata calcolata l'ultima mappa dei guasti e ricalcolarla solo se è passato molto tempo).

Per questo servizio non è richiesta autenticazione.

Schermo distributore (che si ritiene già loggato automaticamente)

1. Pagina principale in cui vengono mostrati nome utente e credito dell'utente connesso (al momento mettete dati fittizi, in futuro verranno caricati dinamicamente) e vi è l'interfaccia per effettuare gli acquisti di bevande calde e selezionare la quantità di zucchero.
2. Con una funzione JavaScript il client periodicamente interroga una URL che comunica i dati dell'utente connesso alla macchina o l'indicazione che nessun utente è connesso. In futuro sarà una componente di backend che interroga un DB al momento sarà un file XML o JSON che contiene le informazioni dell'utente connesso.
3. Visualizzate i dati dell'utente connesso e mostrate quindi la schermata per l'erogazione delle bevande. A ogni erogazione visualizzate un alert che indica di quando è diminuito il credito e aggiornate la visualizzazione del credito sulla pagina.
4. Scrivete il controller che risponde al poll del distributore e che ritorna i dati dell'eventuale utente connesso.
5. Scrivete il controller che risponde alle richieste dell'utente di erogazione bevanda (essenzialmente deve verificare se l'utente loggato ha credito a sufficienza e diminuirlo in caso di erogazione)
6. Aggiungete al front-end una chiamata asincrona ogni 60 secondi ("heartbeat") al servizio di monitoraggio per segnalare che il distributore è attivo.

Cliente

1. Pagina di login
2. Pagina di registrazione

3. Pagina principale in cui vengono mostrati nome utente e credito (al momento mettete dati fittizi, in futuro verranno caricati dinamicamente al login) e vi è l'interfaccia per effettuare la connessione a un distributore (indicando l'ID), scollegarsi, effettuare una ricarica.
4. Modificate le vostre pagine di login e registrazione in modo che, indipendentemente dai dati inseriti (che devono però essere stati validati staticamente con le funzionalità di HTML5) vi portano alla pagina principale. I dati dell'utente loggato, che in futuro riceverete dal backend a seguito del login, li leggerete dallo stesso file XML o JSON che avete creato al punto precedente.
5. Attivate i pulsanti di connessione o disconnessione che al momento visualizzeranno semplicemente un alert col messaggio "Connesso al distributore xxx" o "Disconnesso dal distributore xxx" ma che in futuro invieranno la comunicazione al backend. Analogamente fate in modo che la ricarica vi dia un alert che conferma l'avvenuta ricarica e aggiornate la visualizzazione del credito a schermo.
6. Scrivete il controller che in base allo username inserito (indipendentemente dalla password inserita) vi porta alla pagina principale che mostra i dati dell'utente. L'autenticazione la farete nella consegna finale con Spring security.
7. Scrivete il controller che gestisce la registrazione (verifica che l'utente non esiste e salva i dati sul DB).
8. Scrivete i controller per la connessione/disconnessione dal distributore.
9. Scrivete il controller per la ricarica del credito.

Addetto manutenzione

1. Pagina di login
2. Pagina principale in cui si può inserire un ID distributore e tramite un pulsante viene caricato lo stato di quel distributore (livelli forniture, eventuali guasti).
3. Modificate la pagina di login in modo che vi porti alla pagina principale indipendentemente da login e password inseriti (che devono però essere stati validati staticamente con le funzionalità di HTML5).
4. Modificate la pagina principale in modo che dopo aver inserito l'ID della macchina e premuto il pulsante i dati vengano caricati con una richiesta JavaScript del file XML dell'assignment precedente. Nel file XML che richiederete ci sono più distributori, voi dovete visualizzare i dati relativi al distributore di cui il manutentore ha inserito l'ID o un messaggio di errore se l'ID non è presente nel file XML.
5. Scrivete il controller che in base allo username inserito (indipendentemente dalla password inserita) vi porta alla pagina principale che mostra i dati del manutentore. L'autenticazione la farete nella consegna finale con Spring security.
6. Scrivete un controller che restituisce uno stream XML con i dati di tutti i distributori presenti nel DB nello stesso formato usato nell'assignment precedente.
7. Aggiungete la funzionalità sul front-end per segnalare l'avvenuto ripristino dei livelli forniture e per cambiare lo stato del distributore (attivo/in manutenzione) e le corrispondenti funzionalità sul back-end.

Gestore del sistema

1. Pagina di login
2. Pagine per aggiunta e rimozione di distributori e di addetti manutenzione
3. Funzionalità di attivazione/disattivazione distributori
4. Possibilità di leggere lo stato di tutte le macchine
5. Modificate la pagina di login in modo che vi porti alla pagina principale indipendentemente da login e password inseriti (che devono però essere stati validati staticamente con le funzionalità di HTML5).
6. Create un file XML contenente i dati degli addetti alla manutenzione (non è necessario fare anche uno schema XSD). Modificate la vostra pagina di aggiunta/rimozione addetti alla manutenzione in modo che con una richiesta JavaScript legga i dati dal file XML e li visualizzi in forma tabellare. Utilizzando JavaScript simulate le funzionalità di eliminazione e aggiunta di un

- addetto aggiornando opportunamente la tabella a schermo e visualizzando degli alert (che in futuro verranno sostituiti con le comunicazioni al backend delle modifiche da apportare al DB).
7. Fate lo stesso per la pagina gestione distributori caricando i dati iniziali dal file XML dell'esercitazione precedente.
 8. Aggiungete alla pagina in cui potete leggere lo stato delle macchine funzionalità di base di ricerca. Il requisito minimo è la possibilità di visualizzare tutti i distributori e il distributore di cui specificate l'ID ma se avete altre informazioni nel file XML che potrebbero essere usati per una ricerca sensata provate a implementare altri criteri di ricerca. In ogni caso la ricerca avverrà sul frontend, voi caricate l'intero file XML e, in base alla richiesta dell'utente, visualizzate le informazioni che soddisfano i requisiti.
 9. Scrivete il controller che in base allo username inserito (indipendentemente dalla password inserita) vi porta alla pagina principale del gestore. L'autenticazione la farete nella consegna finale con Spring security.
 10. Scrivete il controller che restituisce uno stream XML con gli addetti alla manutenzione secondo il formato dell'assignment precedente.
 11. Scrivete i controller per gestire sul DB eliminazione e aggiunta di un addetto alla manutenzione (le chiamate ai controller sostituiranno gli alert dell'assignment precedente).
 12. Scrivete i controller per gestire sul DB eliminazione e aggiunta dei distributori (le chiamate ai controller sostituiranno gli alert dell'assignment precedente).

NOTE AGGIUNTIVE

Le viste per distributore, cliente e addetto manutenzione devono essere single page (ad eccezione di login e registrazione che possono essere separate) mentre per il gestore siete liberi di fare single page o multi page.

Scrivete infine un XML Schema per rappresentare le informazioni di stato di tutte le macchine e un file XML di esempio con dati fittizi che valida rispetto allo schema di cui sopra.

IMPORTANTE

L'intera applicazione a questo punto saranno due progetti, quello principale Spring Boot e quello di monitoraggio JakartaEE. La consegna dell'assignment avverrà mediante condivisione di una cartella contenente i due progetti e i file con le specifiche dei database.
